

1_Supervised Learning with Sci-kit Learn(1).ipynb

Cell 1 (markdown)

<h1>CHAPTER:01 CLASSIFICATION</h1>

Cell 2 (code)

```
#load dataset
#import numpy
# Import KNeighborsClassifier
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.neighbors import

churn_df=pd.read_csv("telecom_churn_clean.csv")
churn_df.head()
```

Cell 3 (code)

```
# Create arrays for the features and the target variable
y = churn_df["churn"].values
X = churn_df[["account_length", "customer_service_calls"].values

# Create a KNN classifier with 6 neighbors
knn = KNeighborsClassifier(n_neighbors=5)

# Fit the classifier to the data
knn.fit(X, y)

X_new = np.array([[30.0, 17.5],
[107.0, 24.1],
[213.0, 10.9]])

# Predict the labels for the X_new
y_pred = knn.predict(X_new)

# Print the predictions for X_new
print("Predictions: {}".format(y_pred))
```

Cell 4 (code)

```
# Create arrays for the features and the target variable
y = churn_df["churn"].values
X = churn_df[["account_length", "customer_service_calls"].values

# Create a KNN classifier with 6 neighbors
knn = KNeighborsClassifier(n_neighbors=6)

# Fit the classifier to the data
knn.fit(X, y)
```

```

X_new = np.array([[30.0, 17.5],
[107.0, 24.1],
[213.0, 10.9]])

# Predict the labels for the X_new
y_pred = knn.predict(X_new)

# Print the predictions for X_new
print("Predictions: {}".format(y_pred))

```

Cell 5 (code)

```

#Splitting the data, in order to avoid underfitting and overfitting

# Import the module
from sklearn.model_selection import train_test_split

X = churn_df.drop("churn", axis=1).values
y = churn_df["churn"].values

# Split into training and test sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42, stratify=y)
knn = KNeighborsClassifier(n_neighbors=5)

# Fit the classifier to the training data
knn.fit(X_train,y_train)

# Print the accuracy
print(knn.score(X_test, y_test))

```

Cell 6 (code)

```

from sklearn.svm import SVC
from sklearn.naive_bayes import GaussianNB
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score

# 1. SVM
svm_model = SVC()
svm_model.fit(X_train, y_train)
svm_pred = svm_model.predict(X_test)
print("SVM Accuracy:", accuracy_score(y_test, svm_pred))

# 2. Naive Bayes
nb_model = GaussianNB()
nb_model.fit(X_train, y_train)
nb_pred = nb_model.predict(X_test)
print("Naive Bayes Accuracy:", accuracy_score(y_test, nb_pred))

# 3. Decision Tree
dt_model = DecisionTreeClassifier()
dt_model.fit(X_train, y_train)

```

```
dt_pred = dt_model.predict(X_test)
print("Decision Tree Accuracy:", accuracy_score(y_test, dt_pred))
```

Cell 7 (code)

#LOOP TO TEST DIFFERENT K VALUES:

```
# Create neighbors
neighbors = np.arange(1, 13)
train_accuracies = {}
test_accuracies = {}

for neighbor in neighbors:
    # Set up a KNN Classifier
    knn = KNeighborsClassifier(n_neighbors=neighbor)

    # Fit the model
    knn.fit(X_train, y_train)

    # Compute accuracy
    train_accuracies[neighbor] = knn.score(X_train, y_train)
    test_accuracies[neighbor] = knn.score(X_test, y_test)
print(neighbors, '\n', train_accuracies, '\n', test_accuracies)
```

Cell 8 (code)

```
# Add a title
plt.title("KNN: Varying Number of Neighbors")

# Plot training accuracies
plt.plot(neighbors, train_accuracies.values(), label="Training Accuracy")

# Plot test accuracies
plt.plot(neighbors, test_accuracies.values(), label="Testing Accuracy")

plt.legend()
plt.xlabel("Number of Neighbors")
plt.ylabel("Accuracy")

# Display the plot
plt.show()
```

Cell 9 (markdown)

<H1>CHAPTER:02 REGRESSION</H1>

Cell 10 (code)

```
import numpy as np
import pandas as pd
# Create X from the radio column's values
sales_df=pd.read_csv("advertising_and_sales_clean.csv")
X = sales_df["radio"].values

# Create y from the sales column's values
```

```

y = sales_df["sales"].values

# Reshape X
X = X.reshape(-1,1)

# Check the shape of the features and targets
print(X.shape, y.shape)

# Import LinearRegression
from sklearn.linear_model import LinearRegression

# Create the model
reg = LinearRegression()

# Fit the model to the data
reg.fit(X,y)

# Make predictions
predictions = reg.predict(X)

print(predictions[:5])

#VISUALIZE RESULTS

# Import matplotlib.pyplot
import matplotlib.pyplot as plt

# Create scatter plot
plt.scatter(X, y, color="blue")

# Create line plot
plt.plot(X, predictions, color="red")
plt.xlabel("Radio Expenditure ($)")
plt.ylabel("Sales ($)")

# Display the plot
plt.show()

```

Cell 11 (code)

```

# Create X and y arrays
X = sales_df.drop("sales", axis=1)
y = sales_df["sales"]
X = pd.get_dummies(X, drop_first=True)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

# Instantiate the model
reg = LinearRegression()

# Fit the model to the data
reg.fit(X_train, y_train)

# Make predictions
y_pred = reg.predict(X_test)

```

```

print("Predictions: {}, Actual Values: {}".format(y_pred[:2], y_test[:2]))

# Import mean_squared_error
from sklearn.metrics import mean_squared_error

# Compute R-squared
r_squared = reg.score(X_test, y_test)

# Compute RMSE
rmse = mean_squared_error(y_test, y_pred)

# Print the metrics
print("R^2: {}".format(r_squared))
print("RMSE: {}".format(rmse))

```

Cell 12 (code)

```

#Regularized Regression, sales_df
sales_df=pd.read_csv("advertising_and_sales_clean.csv")
X = sales_df["radio"].values

# Create y from the sales column's values
y = sales_df["sales"].values

# Reshape X
X = X.reshape(-1,1)

# Import Ridge
from sklearn.linear_model import Ridge
alphas = [0.1, 1.0, 10.0, 100.0, 1000.0, 10000.0]
ridge_scores = []
for alpha in alphas:

# Create a Ridge regression model
ridge = Ridge(alpha=alpha)

# Fit the data
ridge.fit(X_train, y_train)

# Obtain R-squared
score = ridge.score(X_test, y_test)
ridge_scores.append(score)
print(ridge_scores)


# Import Lasso
from sklearn.linear_model import Lasso

# Instantiate a lasso regression model
lasso = Lasso(alpha=0.3)

# Fit the model to the data
lasso.fit(X,y)

```

```
# Compute and print the coefficients
lasso_coef = lasso.coef_
print(lasso_coef)
```

Cell 13 (markdown)

<H1>CHAPTER #03 FINE TUNE YOUR MODEL</H1>

Cell 14 (code)

```
#diabetes_Df, split into train and test

diab_df=pd.read_csv("diabetes_clean.csv")
X=diab_df.drop("diabetes",axis=1)
y=diab_df["diabetes"]
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

# Import confusion matrix
from sklearn.metrics import classification_report, confusion_matrix

knn = KNeighborsClassifier(n_neighbors=6)

# Fit the model to the training data
knn.fit(X_train, y_train)

# Predict the labels of the test data: y_pred
y_pred = knn.predict(X_test)

# Generate the confusion matrix and classification report
print(confusion_matrix(y_test, y_pred))
print(classification_report(y_test, y_pred))
```

Cell 15 (code)

```
# Import GridSearchCV
from sklearn.model_selection import GridSearchCV

# Set up the parameter grid
param_grid = {"alpha": np.linspace(0.00001, 1, 20)}

# Instantiate lasso_cv
lasso_cv = GridSearchCV(lasso, param_grid, cv=kf)

# Fit to the training data
lasso_cv.fit(X_train, y_train)
print("Tuned lasso paramaters: {}".format(lasso_cv.best_params_))
print("Tuned lasso score: {}".format(lasso_cv.best_score_))
```

Cell 16 (code)

```
# Create the parameter space
params = {"penalty": ["l1", "l2"],
"tol": np.linspace(0.0001, 1.0, 50),
```

```

"C": np.linspace(0.1, 1.0, 50),
"class_weight": [{"balanced", {0:0.8, 1:0.2}}]

# Instantiate the RandomizedSearchCV object
logreg_cv = RandomizedSearchCV(logreg, params, cv=kf)

# Fit the data to the model
logreg_cv.fit(X_train, y_train)

# Print the tuned parameters and score
print("Tuned Logistic Regression Parameters: {}".format(logreg_cv.best_params_))
print("Tuned Logistic Regression Best Accuracy Score: {}".format(logreg_cv.best_score_))

```

Cell 17 (markdown)

<H1>CHAPTER:04: PREPROCESSING AND PIPELINING</H1>

Cell 18 (code)

```

#Use Music DF
# Print missing values for each column
print(music_df.isna().sum().sort_values())

# Print missing values for each column
print(music_df.isna().sum().sort_values())

# Remove values where less than 5% are missing
music_df = music_df.dropna(subset=["genre", "popularity", "loudness", "liveness", "tempo"])

# Convert genre to a binary feature
music_df["genre"] = np.where(music_df["genre"] == "Rock", 1, 0)

print(music_df.isna().sum().sort_values())
print("Shape of the `music_df`: {}".format(music_df.shape))

```

Cell 19 (code)

```

# Import modules
from sklearn.impute import SimpleImputer
from sklearn.pipeline import Pipeline

# Instantiate an imputer
#By default it uses mean, we can use strategy argument
imputer = SimpleImputer()

# Instantiate a knn model
knn = KNeighborsClassifier(n_neighbors=3)

# Build steps for the pipeline
steps = [("imputer", imputer),
("knn", knn)]

steps = [("imputer", imp_mean),

```

```

("knn", knn)]

# Create the pipeline
pipeline = Pipeline(steps)

# Fit the pipeline to the training data
pipeline.fit(X_train,y_train)

# Make predictions on the test set
y_pred = pipeline.predict(X_test)

# Print the confusion matrix
print(confusion_matrix(y_test, y_pred))

```

Cell 20 (code)

```

#Scaling the Data, using dataset containing genre column

# Import StandardScaler
from sklearn.preprocessing import StandardScaler

# Create pipeline steps
steps = [("scaler", StandardScaler()),
("lasso", Lasso(alpha=0.5))]

# Instantiate the pipeline
pipeline = Pipeline(steps)
pipeline.fit(X_train, y_train)

# Calculate and print R-squared
print(pipeline.score(X_test, y_test))

# Build the steps
steps = [("scaler", StandardScaler()),
("logreg", LogisticRegression())]
pipeline = Pipeline(steps)

# Create the parameter space
parameters = {"logreg__C": np.linspace(0.001, 1.0, 20)}
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=21)

# Instantiate the grid search object
cv = GridSearchCV(pipeline, param_grid=parameters)

# Fit to the training data
cv.fit(X_train, y_train)
print(cv.best_score_, "\n", cv.best_params_)

```

Cell 21 (code)

```

#Evaluate Multiple Models

models = {"Linear Regression": LinearRegression(), "Ridge": Ridge(alpha=0.1), "Lasso":

```



```

Lasso(alpha=0.1)}
results = []

# Loop through the models' values
for model in models.values():
    kf = KFold(n_splits=6, random_state=42, shuffle=True)

# Perform cross-validation
cv_scores = cross_val_score(model, X_train, y_train, cv=kf)

# Append the results
results.append(cv_scores)

# Create a box plot of the results
plt.boxplot(results, labels=models.keys())
plt.show()

# Import mean_squared_error
from sklearn.metrics import mean_squared_error

for name, model in models.items():

# Fit the model to the training data
model.fit(X_train_scaled, y_train)

# Make predictions on the test set
y_pred = model.predict(X_test_scaled)

# Calculate the test_rmse
test_rmse = mean_squared_error(y_test, y_pred, squared=False)
print("{} Test Set RMSE: {}".format(name, test_rmse))

```

Cell 22 (code)

```

#MUSIC DF
# Create models dictionary
models = {"Logistic Regression": LogisticRegression(), "KNN": KNeighborsClassifier(), "Decision
Tree Classifier": DecisionTreeClassifier()}
results = []

# Loop through the models' values
for model in models.values():

# Instantiate a KFold object
kf = KFold(n_splits=6, random_state=12, shuffle=True)

# Perform cross-validation
cv_results = cross_val_score(model, X_train_scaled, y_train, cv=kf)
results.append(cv_results)
plt.boxplot(results, labels=models.keys())
plt.show()

```

Cell 23 (code)

```
# Create models dictionary
models = {"Logistic Regression": LogisticRegression(), "KNN": KNeighborsClassifier(), "Decision
Tree Classifier": DecisionTreeClassifier()}
results = []

# Loop through the models' values
for model in models.values():

# Instantiate a KFold object
kf = KFold(n_splits=6, random_state=12, shuffle=True)

# Perform cross-validation
cv_results = cross_val_score(model, X_train_scaled, y_train, cv=kf)
results.append(cv_results)
plt.boxplot(results, labels=models.keys())
plt.show()
```